Method Override

Implementing the edit feature

Two Requests to Implement the Edit feature

Implementing the edit feature requires two requests.

1. Users use `http://localhost:5500/edit/11` to edit a post (ID 11 as an example).
2. Server returns an HTML page to edit the post 11.
3. Users update the post, and send it back again to the server for updating the post.

We need to make a PUT request to update a post, as a POST request is used to make a new post; this is the recommended and standard REST style.

Issues of Making the PUT request

However, an HTML form cannot make a PUT request (forms support only GET and POST); there are two solutions.

1. Use POST request with edit endpoint: simple but not recommended for REST.
2. Use method override so that the server can treat the POST request as a PUT request.

Implementing Edit Feature

GET Request + edit endpoint with ID

This is the code that processes the ID (11 in the example) when the user visits `http://localhost:5500/edit/11`.

```
app.get('/edit/:id', async function (req, resp) {
  try {
    const posts = db.collection(POSTS)
    const res = await posts.findOne({ _id: parseInt(req.params.id, 10) });
    if (res != null) {
      resp.render('edit.ejs', { data: res });
    } else {
      resp.status(404).send({ error: 'Post not found' });
    }
  } catch (error) {
    console.error(error);
    resp.status(500).send({ error: 'Error from db.collection().findOne()' });
  }
});
```

edit.ejs

```
<form action="/edit?_method=PUT" method="POST">
  <input type="hidden" value="<%= data._id %>" name="id">
  <div class="form-group">
    <label>Update: What to do today</label>
    <input type="text" class="form-control" name="title" value="<%= data.title %>" />
  </div>
  <div class="form-group">
    <label>Update: Date</label>
    <input type="text" class="form-control" name="date" value="<%= data.date %>" />
  </div>
  <button type="submit" class="btn btn-outline-secondary">Submit</button>
</form>
```

A form can use only the GET or POST method, so we use POST and append `?_method=PUT` to the `/edit` endpoint.

Also note the quotes in `value="<%= data.title %>"`: without them, a title with spaces would be cut at the first space.

Method Override Middleware

We can use the method override middleware to transform this POST request with `?_method=PUT` into the PUT request.

```
const methodOverride = require('method-override')
app.use(methodOverride('_method'))
```

In this code, we tell the middleware to look at the `_method` query parameter: the request method is replaced by its value (in this example, PUT).

1. Browser sends a POST request:

```
POST /edit?_method=PUT
content: id=11&title=NewTitle
```

2. `methodOverride` checks `_method`:

```
app.use(methodOverride('_method'));
```

If the request has `_method=PUT`:

3. It replaces the HTTP method internally:

```
POST → PUT
```

4. Express receives:

```
PUT /edit
```

PUT request + edit endpoint

Then Express runs this handler, which updates the post and redirects to `/list`.

```
app.put('/edit', async function (req, resp) {
  try {
    const posts = db.collection(POSTS);

    await posts.updateOne(
      { _id: parseInt(req.body.id, 10) },
      { $set: { title: req.body.title, date: req.body.date } }
    );

    console.log('app.put.edit: Update complete');
    resp.redirect('/list');    // go back to the list page
  } catch (e) {
    console.error(e);
    resp.status(500).send('Update error');
  }
});
```